

Design Principles for Research Software

October 2, 2026

Contents

About these guidelines	3
Ladder of Research Software	3
Normative language	3
Library code and workflow code	4
Packages	5
Terminology	5
I All code	5
1 Guiding priorities	5
1.1 Build the right thing before building the thing right	6
1.2 Get your priorities right	6
1.3 Better loud than wrong	6
1.4 Pay as you grow	7
1.5 KISS: Keep it simple, stupid	7
2 Scope	7
2.1 Own only the core domain	7
2.2 Code is for life, not just for Christmas	8
3 Code structure and abstraction	8
3.1 Derive abstractions from applications, not applications from abstractions	8
3.2 DRY: Don't repeat yourself	9
3.3 No magic numbers	9
3.4 Duplicate code is cheaper than the wrong abstraction	9
4 API design	9
4.1 Write the API first	9
4.2 Name axes, units, and conventions	10
4.3 Numerics and defaults are part of the API	10
4.4 Design stable APIs	10
4.5 Document the method, not the implementation	10
5 Explicit behavior	11
5.1 Do what it says on the tin	11
5.2 Don't guess	11
5.3 Do not mutate caller data	11
5.4 Raise, warn, or stay silent	11
6 Input validation and errors	12
6.1 Garbage in, useful error out	12
6.2 Missing values and NaN are explicit	12
6.3 Wear belt and braces	13

7	Numerical correctness	13
7.1	Prefer numerically stable formulations	13
7.2	Report conditioning and convergence	13
8	Testing and reproducibility	14
8.1	Testing is not optional	14
8.2	Test invariants, not only examples	14
8.3	Size matters	14
8.4	Reproducibility: the same answer under stated conditions	14
8.5	Compare floats with tolerance	14
8.6	Flaky tests cry wolf	15
8.7	Make every result traceable to a version	15
II	Workflow code	15
9	Steps and their files	15
9.1	Out of sight, out of date	15
9.2	Nothing lost in translation	16
9.3	All or nothing	16
10	Environments and tests	16
10.1	Pin what you run	16
10.2	Test the whole workflow on a small dataset	17
III	Library code	17
11	Structure	17
11.1	Design modules around clear responsibilities	17
11.2	No code at large	17
11.3	Avoid plotting in core functionality	18
12	Interfaces	18
12.1	Operate on data objects, not files	18
12.2	Make the common case easy, and the uncommon case possible	18
12.3	The library follows the caller's data types	18
12.4	Keep validation proportionate	19
13	No side effects	19
13.1	Do not touch global state	19
13.2	No surprise cost	20
13.3	Importing the package has no side effects	20
IV	Other code forms	20
14	Graphical applications	20
15	Web applications and services	21
16	Notebooks	21
17	Simulation codes	22
18	Plugins	22

About these guidelines

These guidelines are for research software: all code written for a research purpose, in academia or in industry, from the script a PhD student writes for one analysis to the library or workflow that a community or a company relies on. They are independent of programming language and scientific field.

Research software stands on a ladder, from code that only its author runs to software that others build on and base publications and decisions on. Most software starts at the bottom, and much of it should stay there. The principles in these guidelines are what it takes to climb. Usually the science is known; the task is to make it reliable, automated, and fit to run at scale.

Ladder of Research Software

Research software stands on one of four rungs (Table 1). Which one depends on the role of the software: how many people use it, how often and for how long, on how much data, and what a wrong result would cost. The higher the rung, the more depends on the software, and the more of these guidelines it has to follow.

Table 1: The Ladder of Research Software.

Rung	Who uses it	Examples	What it takes
1 Personal	Its author, for one analysis	The script a PhD student writes to try out an idea	Nothing yet; the principles already make the code easier to trust
2 Shared	Its author again and again, or colleagues	A script passed around a group; a workflow that a lab reruns for every new batch of samples	Every <i>must</i> : the floor; the <i>shoulds</i> where they pay off
3 Released	People its authors do not know and cannot support one by one	A published package; a workflow shared between institutions; a tool used across the departments of a company	The floor and, as a rule, the <i>shoulds</i> ; documentation that users can work from alone; a stability commitment
4 Infrastructure	Others who build on it or base decisions on it	A library that many packages depend on; a workflow that processes every sample of a core facility; software whose results enter a drug approval or an emission limit	The floor and, as a rule, the <i>shoulds</i> ; testing and documentation as thorough as the cost of a wrong result demands; long stability commitments and controlled changes

The requirements marked *must* are the floor from the second rung on; most of them cost little and prevent wrong results. What rises with each rung is everything above the floor: how thoroughly the *shoulds* are followed, the depth of testing and documentation, the effort put into performance, and the length of the stability commitment (Subsection 4.4).

A high cost of a wrong result can put software on a higher rung than its number of users suggests: a script that only its author runs, but whose numbers go into a regulatory submission, is not personal software. When and how to climb is the subject of Subsection 1.4.

Normative language

The guidelines distinguish requirements from defaults. The following words carry a fixed meaning throughout, in the spirit of RFC 2119:

must, must not A requirement for all software from the second rung of the [Ladder of Research Software](#) on. Code that violates it is a defect and is fixed, not justified.

should, should not A strong default. A deviation is acceptable only for a concrete reason, which is stated where the deviation occurs, for example in a code comment or in the review. The rung of the software on the ladder is such a reason; it is stated once for the whole package, for example in its documentation, rather than at each deviation.

may Permitted, but not required.

Plain imperatives such as “prefer” and “avoid” carry the weight of *should*. When two rules pull in different directions, the priorities in Subsection 1.2 decide.

Library code and workflow code

The code of research software does one of two jobs, and the Hollywood principle tells them apart: *don't call us, we'll call you*.

Library code is a collection of reusable building blocks, such as functions, classes, and data types, that callers use in their own code. It does not decide what happens or in which order: the caller owns the program and decides. Library code should provide composable primitives that users call from their own scripts, notebooks, workflows, and host-stack tools, and should not take over the main program, the solver or simulation loop, the data model, or the analysis workflow. Where the method needs code from the user, such as an objective function, a model, or the right-hand side of a differential equation, accept it as an ordinary function argument. Requiring users to subclass package classes or to register code with a package-managed lifecycle turns library code into a framework on top of the host stack.

Library code is code called by other code. It provides building blocks and leaves the decision of what runs, and in which order, to its caller.

Workflow code follows the Hollywood principle: it owns the program and calls other code to do the work. A workflow is a defined sequence of steps that carries out a task from start to finish. It encodes the order of the steps and the inputs and outputs of each, and often branching, retries, parallel execution, and scheduling; the user mostly supplies inputs and configuration. Workflow definitions, for example for Snakemake, Nextflow, or Make, and the scripts of their steps are workflow code. So is the main function of a command-line tool, which reads the arguments and the input, calls library code, and writes the output: each command is a small workflow.

Workflow code is code that owns the program and calls other code. It decides what runs and in which order.

In a kitchen, library code is the knives, the pans, and the techniques; workflow code is the recipe, which says to chop this, then sauté that for five minutes, then combine.

The scientific logic of a package should be library code, where it can be tested, reused, and scaled without running the whole workflow. Workflow code may contain a piece of science while that piece is short, used in only one place, and tested by running the workflow code on a small input (Subsection 10.2). The piece should move into library code once a second step or code outside the workflow needs it (Subsection 3.2), once it needs tests of its own, or once it grows beyond what a reader can check at a glance. Wherever the science lives, the rules about the science apply to it.

Workflow code owns its program, and with it the program's output, files, and process. Rules that restrict library code, such as those on printing, files, and global state, therefore do not apply to it.

The principles are grouped accordingly. Part [I](#) applies to all code; where a principle works differently for the two kinds of code, it says how. Part [II](#) adds the principles that apply only to workflow code, and Part [III](#) those that apply only to library code. Part [IV](#) describes other forms of research software, such as graphical applications and notebooks, and what each gains and loses compared with library and workflow code.

Packages

Library code and workflow code describe what code does. The package describes how it is shipped.

A **package** is the unit of distribution: what is installed, versioned, and shared. It may contain library code, workflow code, or both, and it is not a synonym for a library.

Mixing the two kinds of code is common. A package may offer library functions for users who want fine control together with a command that chains them into a standard analysis, or ship workflow definitions as files next to its library code. A command-line tool that does one job, such as sorting sequence alignments or regridding a climate field, is therefore library code behind a thin layer of workflow code, while a tool that runs a whole analysis by calling other tools in a fixed order is a workflow, even when it is started like any other command.

When a package contains both, workflow code calls library code, never the other way round. Library code should not depend on the workflow code of its package (Subsection [11.1](#)), so that users can reuse the building blocks in their own workflows without being forced into the package's own.

Terminology

Package The unit of distribution: what is installed, versioned, and shared ([Packages](#)).

Library code Code called by other code ([Library code and workflow code](#)).

Workflow code Code that owns the program and calls other code ([Library code and workflow code](#)).

Host stack The software ecosystem the user already works in: the programming language, its standard array and data-frame types, its established numerical, statistical, and plotting libraries, and the user's own workflow tools. Examples are the scientific Python ecosystem, R with its package repositories, and Julia with its package ecosystem. The host stack sets the conventions for data types, randomness, warnings, logging, and packaging that the package follows.

Entry point Anything a caller invokes to hand inputs to the package and get a result: a user-facing function, a command of a command-line tool, a workflow, or a step of a workflow.

User-facing function Any function, class, or method that is documented for users, including low-level primitives. Together, the user-facing functions form the public API; everything else is internal.

Caller The code that calls an entry point, and by extension the user who wrote or runs it.

Part I

All code

1 Guiding priorities

1.1 Build the right thing before building the thing right

Establish that the overall approach is correct before investing in optimization, testing, documentation, or polish. Effort spent in the wrong direction is not partially recovered; it is lost in full, and the better the execution, the greater the loss.

Incremental improvement converges on the nearest local optimum, not on the best achievable one. A design can therefore become steadily better while remaining fundamentally misplaced, and no amount of further refinement will reveal this, because refinement measures progress against the current approach rather than against the alternatives to it.

Quality and speed multiply direction rather than substitute for it. Executing a wrong approach quickly and well only arrives sooner at a place that must be abandoned. Refinement also entrenches: the more effort a design has absorbed, the harder it becomes to question, so the assessment should happen early and be repeated deliberately.

Identify which decisions are reversible and which are not. Reversible decisions can be made quickly and corrected later. Irreversible decisions, such as the core abstraction and the public interface, deserve the effort that would otherwise be spent perfecting details, because they determine whether that effort is worth anything at all.

1.2 Get your priorities right

When goals conflict, rank them as follows, and let the higher one win:

1. **Correctness.** The results are right and reproducible; a result that cannot be reproduced cannot be checked (Subsection 8.4).
2. **Clarity.** Code, interface, and behavior can be understood. Code that cannot be understood hides its errors.
3. **Testability.** Correctness can be demonstrated, not merely claimed.
4. **Maintainability.** The package stays correct and clear as it changes.
5. **Performance.** The package is fast enough for the data sizes of real applications (Subsection 8.3).
6. **Features.** More functionality, options, and code are a bonus, never a substitute for the priorities above.

The same ranking resolves conflicts between the rules in these guidelines: when two rules pull in different directions, the rule that serves the higher priority wins. Prefer a small amount of reliable functionality over a large amount of unreliable functionality; code that is produced quickly but is hard to understand, test, or trust only adds to the burden of the package.

1.3 Better loud than wrong

When something goes wrong, the package must fail visibly rather than return a result that looks valid but cannot be trusted. Given the choice between an error and a result that may be wrong, choose the error.

The two outcomes are not symmetric. A failure is annoying and can cost time, but it is visible and its cost is bounded: the user sees it, fixes the cause, and runs again. A silently wrong result looks exactly like a correct one. It flows into further analyses, figures, publications, and decisions, and nobody looks for the error, because nothing suggests that there is one. A failure costs convenience; a silently wrong result costs correctness, which ranks first (Subsection 1.2).

Silent failures are rarely designed. They usually arise from code that tries to be helpful or robust. Typical sources are:

- Catching an error and returning a default value, an empty result, or NaN in place of the result.
- Falling back to a different algorithm, approximation, or data source when the intended one fails or an optional dependency is missing.
- Letting integers overflow, clipping or truncating values to fit a type or range, or broadcasting arrays whose shapes do not match.
- Dropping records that find no match in a join or alignment.
- Continuing after a step has failed, so that an incomplete output looks complete.

Each of these turns a failure the user would have seen into a wrong result the user will not see. Let errors that cannot be handled correctly propagate, and make every fallback, coercion, and omission explicit.

Being loud does not mean giving up at the first problem. A workflow that processes thousands of samples may skip one that fails, and an iterative method may return a result that did not converge (Subsection 7.2), provided the degradation is explicit: it is documented or chosen by the caller, the result records what is missing or uncertain, and the failure is reported through the channels of Subsection 5.4. Workflow code that did not complete what was asked must also say so in its exit status (Subsection 9.3). What must never happen is a degraded result that cannot be told apart from a valid one.

Many rules in these guidelines apply this principle to a particular situation. Where none of them covers a case, the default is the same: when in doubt, fail.

1.4 Pay as you grow

Engineering effort has to pay for itself, and not all research software deserves the same investment. Invest what the rung of the software on the [Ladder of Research Software](#) demands. Building for a higher rung takes effort away from the science; building for a lower one is paid for in wrong results.

Software usually climbs without anyone deciding it, and the first step is the one most often missed: the script that a colleague asks for is already on the second rung. Reassess the role whenever the users, the data, or the stakes change, and climb before new users rely on the software, not after the first wrong result. Not every script needs to climb, and one that stays on the first rung is not a failure.

1.5 KISS: Keep it simple, stupid

Prefer the simplest design that solves the problem well. Simplicity is measured by how much a reader has to hold in mind to understand the code and its behavior: the number of concepts, options, code paths, and special cases, not the number of lines.

Every option multiplies the behaviors that have to be documented and tested, because options interact. Every layer of indirection is a place where a reader has to stop and look elsewhere. Add either only when a concrete application demands it (Subsection 3.1).

Prefer plain, direct code over clever code. When two designs solve the problem equally well, choose the one with fewer concepts. Simple does not mean simplistic: a design that looks simple because it ignores part of the problem, such as missing values or edge cases, is not simple but incomplete.

2 Scope

2.1 Own only the core domain

Custom code belongs only in the distinctive scientific problem that justifies the package. That problem is the core domain. Linear algebra, optimization, numerical integration, automatic differentiation,

parallel execution, file input and output, and plotting are generic capabilities: reuse mature libraries and tools from the host stack instead of rebuilding them.

A package must document, test, and maintain everything it owns. Owning generic work therefore takes effort away from the scientific contribution, and a reimplementaion is usually less tested, less optimized, and less numerically robust than the established one. Implement the core; compose with the host stack for the rest.

2.2 Code is for life, not just for Christmas

Everything the package takes on is a commitment for the lifetime of the software, and that lifetime usually exceeds the involvement of the people who wrote it. Removing something later breaks the users who rely on it (Subsection 4.4). Judge each addition by what it will cost to maintain over years, not by the effort of adding it today, and prefer established, well-understood technology over the newest option.

Features. Every feature adds to what the package must document, test, and maintain (Subsection 2.1) for as long as it lives. Prefer a smaller package with well-tested core functionality over a larger one with many unfinished or poorly supported features.

Dependencies. Each dependency is code the package did not write but still has to install, pin, test against, license, and debug. Prefer the host stack the user already has, and add a dependency only when it owns a job the package must not reimplement and cannot leave to the caller. Choose dependencies that will outlive your own project: widely used, maintained by more than one person, backed by an institution or an active community, and with a history of regular, compatible releases. Avoid dependencies that rest on a single maintainer, thesis, or grant; where a fragile one is unavoidable, isolate it behind a thin internal interface so that it can be replaced. Dependencies needed only for optional functionality, such as plotting, should be optional, so that the core installs without them.

The package itself. Build it to survive its authors. No part of the code should be understood by only one person, and the knowledge needed to maintain it, such as design decisions, conventions, and the release process, belongs in the repository, not in people's heads.

3 Code structure and abstraction

3.1 Derive abstractions from applications, not applications from abstractions

Build for concrete applications that someone actually wants to run. An abstraction invented first, in the hope that useful applications will follow from it, is a guess about demand, and such guesses fail in ways that become visible only once real work is attempted.

Generality that no application requested is speculative generality. It adds options, layers, and special cases that must be documented, tested, and maintained forever while solving no problem anyone has. The cost is paid immediately, the benefit remains hypothetical.

Several genuinely different use cases are usually required before the shared concept becomes visible; the rule of three sets the customary threshold at three. A single use case invites a design fitted too tightly to it, and no use case invites a design fitted to nothing.

Keep the driving applications in the repository as runnable examples or integration tests. They document intent, they fail when the design drifts away from practice, and they force every proposed feature to name the concrete task that justifies it. Software is a practical artifact and can only be judged in use.

3.2 DRY: Don't repeat yourself

Every piece of knowledge should have a single, authoritative representation in the software. DRY is not primarily about code. Knowledge includes rules, assumptions, constants, numerical defaults, conventions such as axes and units, validation logic, data schemas, and the documentation that describes them.

When the same knowledge lives in several places, every change has to find and update all copies consistently. Copies that drift apart produce software that contradicts itself. Typical examples in scientific code are a tolerance hard-coded in several functions, the same input check reimplemented in each user-facing function, a default value repeated in the function signature, its documentation, and the examples, or a threshold set both in the configuration of a workflow and in the script of one of its steps. Define such knowledge once and refer to it everywhere else. Where documentation has to state a value, generate it from the single definition if the tooling permits.

DRY concerns knowledge, not text. Code that looks different can still duplicate the same rule. Conversely, code that looks alike may encode different rules, and merging it is a mistake (see Subsection 3.4).

3.3 No magic numbers

Every numerical constant with a scientific or numerical role, such as a threshold, cutoff, tolerance, iteration limit, or physical constant, must have a name, a unit where it has one (Subsection 4.2), a single definition (Subsection 3.2), and a justification stated next to that definition: a reference, a derivation, or the reason for the choice. In scientific code, an unexplained 3.0 or 0.05 is often an unexamined modeling decision, and a reviewer cannot tell a deliberate choice from an arbitrary one.

If a constant affects the results and users might reasonably want a different value, expose it as a parameter; its default then becomes part of the API (Subsection 4.3). Take physical constants from an authoritative source in the host stack rather than typing them in by hand. Constants whose meaning is evident from the code, such as the 2 in 2π , need no name.

3.4 Duplicate code is cheaper than the wrong abstraction

Avoid premature abstraction. Similar-looking code should be unified only when it represents the same underlying concept or rule (Subsection 3.2). A wrong abstraction hides important differences, increases complexity, and makes future changes harder, because every change has to respect all of its callers at once. Prefer simple duplication until the shared idea is clear and stable.

A common sign of a wrong abstraction is a shared function that accumulates flags and parameters whose only purpose is to switch its behavior for particular callers. When that happens, inline the function back into its callers, remove from each copy what that caller does not need, and extract a new abstraction only once the genuinely shared rule has become visible.

4 API design

4.1 Write the API first

Write the code the user would write before writing the code that makes it work. The calling code is the part that must be good, because it is the part users see, learn, and depend on. Everything behind the interface can be rewritten later; the interface itself cannot. For workflow code, the calling code is the command, the configuration, and the table of samples or inputs that users write.

Designing outward from the implementation produces signatures that expose internal structure rather than user intent. Designing inward from the call site produces the opposite, and reveals awkwardness while it is still free to fix.

Judge a proposed interface by reading realistic usage rather than the definition. Argument order, naming, and defaults that appear reasonable in a signature often read badly in the script that calls them, and that script is the true specification.

An interface that cannot be written cleanly is a signal that the underlying concept is not yet clear. Resolve that before implementing, since implementation effort tends to entrench whatever interface it was built to serve.

4.2 Name axes, units, and conventions

Axis order, such as observations-by-variables versus variables-by-observations or time-major versus space-major, the index base, coordinate systems and reference frames, angle conventions, and physical units are part of the meaning of every array. If they are not in the signature, the documentation, the returned object, or the file that holds the data, users will infer them wrongly. The package must not infer them from the data either (Subsection 5.2).

Do not rely on a comment or an example notebook to carry a convention. Name the axes, state the units, and prefer the labeled data structures of the host stack where they exist. Keep identifiers the caller supplied, such as sample IDs, gene names, station codes, or node labels, intact and aligned with the data through every operation.

4.3 Numerics and defaults are part of the API

Changing a default normalization, initialization, regularization strength, convergence tolerance, iteration limit, ϵ , reduction, or tie-breaking rule can change published numbers. Such changes must be treated as breaking changes of the API, not as internal tweaks. This holds for the defaults of functions, of command-line options, and of workflow configurations alike.

Signatures are not the whole contract. Documented numerical defaults are part of the scientific behavior of the package. If a default has to change, follow the process for breaking changes in Subsection 4.4.

4.4 Design stable APIs

API stability builds user trust, and in science it protects published results. Avoid breaking changes that are not necessary. Keep the public API small, since everything public is a commitment, and mark internal code as private using the host stack's convention. The public API of workflow code is what its users write and read: its command-line options, the keys and defaults of its configuration, the formats of its inputs, and the names and layout of its outputs.

Number releases so that the version communicates compatibility, for example with semantic versioning, and record every change of behavior in a changelog. Changes of numerical behavior, including changes of defaults (Subsection 4.3), must be listed explicitly in the changelog, not summarized as fixes.

When a breaking change is necessary, deprecate before removing: emit a deprecation warning (Subsection 5.4) that names the replacement, keep the old behavior available for a stated period, and document the migration path.

4.5 Document the method, not the implementation

Documentation should describe what a function computes and how to use it, not how the code is organized internally. Clear interfaces allow the implementation to change without breaking user code.

In research software, however, the method is part of the behavior. The documentation of a user-facing function, a command, or a workflow should state the algorithm or estimator, the approximations and assumptions it makes, its numerical defaults, its policy for missing values (Subsection 6.2), and

references to the literature that defines the method; for a workflow, it also names the method and the tool that each step uses. Users need this knowledge to interpret and report their results.

Implementation details, such as loop structure, internal data structures, and private helper functions, do not belong in user documentation. The dividing line is whether a change could alter a scientific result: if it could, it is part of the method and must be documented; if it could not, it is implementation.

5 Explicit behavior

5.1 Do what it says on the tin

An entry point should do what its name and arguments say, and nothing more. Operations that change the data, such as filtering, normalization, sorting, deduplication, or outlier removal, must not be hidden inside an entry point named for a different operation: a user who calls a fitting function should not discover later that the data were also filtered, and a workflow step named for alignment should not also discard reads. Expose such operations as named arguments or configuration options, or provide them as separate functions or steps. Operations that are part of the definition of the method are documented as such (Subsection 4.5).

This is the counterpart of Subsection 5.2: an entry point must neither read more into a call than it states nor do more than it asks.

5.2 Don't guess

The package must not resolve an ambiguity in the user's intent on the user's behalf. A default is not a guess: it is fixed, documented, and independent of the data. A guess applies heuristics to the data to decide what the call means, such as transposing a matrix because of its shape, detecting whether data are already normalized, inferring units or date formats from the values, matching labels approximately, or repairing malformed input. If such an interpretation could change the result, the user has to decide: it is a scientific choice, and it belongs to the user. A guess also makes the meaning of a call depend on the data, so the same code can do different things for different datasets. When the guess is wrong, the result is wrong without looking wrong (Subsection 1.3).

Make the user state the choice through an argument or a configuration option, or raise an error that lists the possible interpretations. Where automatic selection is genuinely useful, offer it as an explicit option, such as a `method` argument with the value `auto`, and record the choice with the result. Heuristics may trigger warnings (Subsection 5.4), but must not change behavior: announcing a guess does not make it the user's choice.

5.3 Do not mutate caller data

An entry point must not overwrite, transpose, densify, drop missing values from, or otherwise rewrite data the caller still holds, unless the user opted into an explicit in-place operation, such as a clearly named in-place function or argument. Silent mutation makes it impossible to know which object or file still holds the original data.

If a transformation is required, library code returns a new object, and workflow code writes a new file; a step must not modify, overwrite, or delete its input files. The default is: leave caller data untouched.

5.4 Raise, warn, or stay silent

Code communicates with its caller through three channels, and each situation calls for exactly one of them:

- **Raise** when the call is invalid, or when continuing would return an untrustworthy result as if it were valid. The user has to act, and execution must not continue as if nothing happened.

- **Warn** when the call is valid but is probably not what the user intended, for example when a heuristic suggests that the data are not what the call assumes (Subsection 5.2), or when the behavior of the call will change in a future version. Deprecations (Subsection 4.4) belong here.
- **Stay silent** when the call is valid and unremarkable.

A warning must not replace an error: if continuing would produce a wrong result, raise (Subsection 1.3). Warnings should name the argument or condition that triggered them and say how to avoid them by changing the call. Avoid emitting the same warning repeatedly, for example inside a loop.

Library code raises and warns through the host stack’s standard mechanisms for errors and warnings, so that users can catch errors, filter warnings, or turn them into errors. It must not print: standard output belongs to the user’s scripts, notebooks, and batch jobs. Progress reports and diagnostic messages fall outside the three channels. They should be opt-in, through an explicit argument or the host stack’s standard logging or messaging facility, so that the user decides whether and where they appear.

Workflow code raises by stopping with an error exit status and a message on the standard error stream (Subsection 9.3), and warns on the standard error stream or in its log. Messages must not go to standard output where it carries data, because they would end up in the data.

6 Input validation and errors

6.1 Garbage in, useful error out

User-facing functions, commands, and workflows must validate their inputs at the point of entry: types, shapes, bounds, units where they are declared, and the semantic constraints of the method. Invalid inputs, unsupported combinations of arguments, and dangerous settings must produce an error at the boundary, not a numerical failure or a wrong result further downstream. Fail loudly and early (Subsection 1.3). A workflow should check its configuration and its table of inputs before its first step runs, so that a typo fails within seconds, not after hours of computation.

Error messages should be specific enough that users can recover without searching through the full documentation: name the argument, state what was expected and what was received, and where possible say how to fix the call.

Do not rely on documentation for correctness. Many users read documentation only partially, skim examples, or rely on auto-completion and error messages while using the package. Documentation explains correct usage; the code must enforce the constraints that correctness depends on. Make misuse difficult and failures obvious.

Implement each check once, in the validation module (Subsection 11.1), and call it from every entry point that needs it (Subsection 3.2). Private functions that are reached only through a validated entry point should not repeat the checks.

6.2 Missing values and NaN are explicit

Each entry point must have a documented policy for missing values, such as NaN, NA, or masked entries: reject, propagate, or mask. Silent imputation, dropping, or filling is a scientific bug, not a convenience (Subsection 1.3): it replaces a missing measurement with a guess (Subsection 5.2).

If the input violates the policy, the entry point must raise an error that names the missing values, rather than fail later with an unrelated numerical error. Checking for missing values is a data check in the sense of Subsection 12.4.

6.3 Wear belt and braces

Tests check the package on the inputs someone thought of, but real data reach paths that no test exercised. A bug that produces a silently wrong result is the worst failure research software can have, so the key invariants of a method should also be checked in the code itself, on every run. Such assertions complement the tests; they do not replace them.

Assert what must hold if the code is correct and whose violation would otherwise go unnoticed, such as a normalization, a conservation law, or the alignment of identifiers after a reordering. These are often the same invariants the tests check (Subsection 8.2). Keep assertions cheap (Subsection 13.2), and do not assert what holds by construction or what validation already guarantees; mistakes of the caller belong to validation (Subsection 6.1).

A failed assertion means the package has a bug. It must raise, never warn, and its message should say so and ask for a report, rather than tell the user to change the call. Assertions must run in every configuration; where the assertion mechanism of the host stack can be disabled, raise explicitly instead.

7 Numerical correctness

7.1 Prefer numerically stable formulations

A formula that is correct in exact arithmetic can be badly wrong in floating point. Choose formulations that remain accurate for the whole range of inputs the package accepts, not only for well-behaved test data. Common patterns include:

- Avoid subtracting nearly equal quantities, which cancels significant digits. Rearrange the formula, or use dedicated functions such as `log1p`, `expm1`, and `hypot`.
- Work in log space for products of many small numbers, such as likelihoods, and compute $\log \sum_i e^{x_i}$ as $m + \log \sum_i e^{x_i - m}$ with $m = \max_i x_i$.
- Solve linear systems instead of forming explicit matrix inverses, and prefer factorizations suited to the structure of the problem.
- Accumulate long sums with care, for example by pairwise or compensated summation, or with a higher-precision accumulator (Subsection 12.3).

Reuse the stable routines of mature libraries rather than reimplementing them (Subsection 2.1). Test behavior at the boundaries of the input range, such as very small and very large magnitudes and nearly singular inputs, as part of the invariant tests of Subsection 8.2.

7.2 Report conditioning and convergence

Distinguish an unstable algorithm from an ill-conditioned problem. An unstable algorithm is a defect of the package and must be fixed. An ill-conditioned problem cannot be solved accurately by any algorithm; the package cannot fix it, but it must not hide it. Where conditioning can be estimated at reasonable cost, check it and report an ill-conditioned problem through the channels of Subsection 5.4.

Iterative methods must not return a result that did not converge as if it had (Subsection 1.3). When the iteration limit is reached or the convergence criterion fails, either raise, or return a result that carries an explicit convergence status and warn. Each iterative function must document which of the two it does, and its result should record diagnostics such as the number of iterations and the final residual.

8 Testing and reproducibility

8.1 Testing is not optional

Scientific code is particularly prone to subtle errors that produce plausible but wrong numbers. Every entry point must have tests, and every release must pass the full test suite. The steps of a workflow are tested by running the whole workflow on a small dataset (Subsection 10.2).

Tests should cover error paths as well as results: invalid inputs must be shown to raise the documented errors (Subsection 6.1). Every fixed bug should gain a regression test that fails without the fix. Test the range of input types the package claims to support, such as sparse, single-precision, or lazy data (Subsection 12.3); a code path that is never exercised is not supported.

Run the test suite automatically on every change, across the versions of the language and of the host-stack libraries that the package supports.

8.2 Test invariants, not only examples

Passing on a few happy-path inputs is a weak scientific test. Prefer invariants: conservation laws, symmetries, known analytic solutions and limiting cases, consistency relations such as counts and normalizations, and agreement with an independent implementation.

Examples are useful, but they do not prove that the method is correct. Encode the scientific claims the software makes and test those claims directly.

8.3 Size matters

A method that works on test data but fails at the scale of real applications solves no problem. Document the time and memory complexity of each entry point in terms of the data size, and benchmark at the sizes of the driving applications (Subsection 3.1), not only on toy inputs. Profile before optimizing, since bottlenecks are rarely where intuition expects them, and track benchmarks to catch performance regressions. Performance never outranks correctness or clarity (Subsection 1.2).

8.4 Reproducibility: the same answer under stated conditions

Results must be reproducible. In these guidelines, reproducibility means obtaining the same scientific answer, within a stated tolerance, from the same inputs, parameters, and package version, on a documented software and hardware stack. It does not mean identical bits forever: floating-point results legitimately vary across linear algebra libraries, processors, accelerators, and compiler settings.

Randomness must be controllable by the caller. Where the host stack supports it, every function that uses random numbers should accept a random number generator or seed argument that controls all randomness within that call. Where the host stack's convention is a single global generator, draw from the generator the caller controls, subject to Subsection 13.1. The behavior when no generator or seed is supplied must be documented. Workflow code takes the seed from its configuration, passes it to every step that uses random numbers, and records it with the results (Subsection 8.7).

Prefer deterministic algorithms. Where nondeterminism is unavoidable, for example because parallel reductions sum in varying order, document it and ensure that results still agree within the stated tolerance.

8.5 Compare floats with tolerance

Tests should compare floating-point results within a tolerance, not bit for bit. Bit-identical reference files treat platform noise as a logic bug and make the test suite fragile.

Assert agreement within the tolerance that defines reproducibility (Subsection 8.4). Use relative tolerances for quantities of varying magnitude and absolute tolerances for quantities near zero.

Tolerances are constants in the sense of Subsection 3.3: derive each from the expected numerical error of the method, rather than adjusting it until the tests pass on one machine.

8.6 Flaky tests cry wolf

A test that fails at random teaches people to ignore failures, and a test suite whose failures are ignored checks nothing. Tests of stochastic methods must therefore fix the random generator (Subsection 8.4), so that every run gives the same outcome.

A fixed generator makes a test repeatable, not convincing: a test that passes for only one seed can hide a bug. Where the claim is statistical, such as an unbiased estimate or a coverage rate, assert it with bounds whose false-failure rate is known and very small, for example a deviation of several standard errors, so that the test would pass for other seeds as well. The bound is a constant in the sense of Subsection 3.3. Never choose a seed because the test passes with it.

8.7 Make every result traceable to a version

A result is reproducible only if the exact software that produced it can be identified and obtained again, whether the result ends up in a publication, a report, or a decision.

The package must expose its version at run time through the host stack's standard mechanism, such as `__version__` in Python or `packageVersion` in R, and a command-line tool through a `--version` option. Every release should be tagged in version control and archived so that the exact version can be retrieved later: in a public archive with a persistent identifier, such as a DOI, for published software, or in the organization's release archive for internal software. Published software should also provide machine-readable citation metadata, such as a `CITATION.cff` file, stating how to cite both the software and the method.

Results should record the package version and the parameters that produced them, including defaults that were not set explicitly: library code records them in the result objects it returns, and workflow code records them in the outputs and run metadata it writes, together with the versions of the tools and libraries it ran (Subsection 10.1). Changes of behavior between versions are traced through the changelog (Subsection 4.4).

Part II

Workflow code

9 Steps and their files

9.1 Out of sight, out of date

Every step must declare everything its outputs depend on: its input files, including reference data, its parameters, and the code and environment that run it. A workflow manager, such as Snakemake, Nextflow, or Make, decides which steps to rerun by checking what it knows about. An input it does not know about can change without triggering a rerun, and the old output stays in place, looking as valid as before.

Such a result is stale, and nothing marks it as stale (Subsection 1.3). A step that reads an annotation file from a hard-coded path keeps its counts from the old annotation after the file has been updated; a script that reads a threshold from an environment variable keeps results computed with the old threshold. Undeclared inputs are the workflow counterpart of hidden global state: the output depends on something the caller cannot see.

Pass parameters through a mechanism the workflow manager tracks; where it tracks none, write them to a file that is declared as an input. A step must also not write files it does not declare as outputs, because a step that reads them may run before they exist or after they have changed. Where an input cannot be tracked, such as a database queried over the network, record its version or the date of the query with the results (Subsection 8.7).

9.2 Nothing lost in translation

Steps hand data to each other through files, and each hand-off is a translation: one step turns data objects into bytes or text, and the next turns them back. The translation must not change the data. Values, their precision, missing values, units, axis and coordinate conventions, and identifiers must arrive in the next step as they left the previous one.

Text formats lose information silently. A floating-point number written with too few digits loses precision. NaN, NA, an empty field, and the string NA may be read as one another or as something else, depending on the reader (Subsection 6.2). Identifiers that look like numbers, such as station codes or sample barcodes, lose their leading zeros, and dates are read in the wrong format. Each of these becomes a plausible wrong result in a later step (Subsection 1.3). Prefer formats that store types, missing values, and metadata, such as Parquet, HDF5, NetCDF, or Zarr, where the host stack supports them. Where a text format is needed, state the number of digits and the encoding of missing values when writing, and the type of every column when reading.

A file passed between steps is an interface, like the signature of a function. Its column names, types, units, and encoding of missing values are knowledge that belongs in one place (Subsection 3.2), such as a schema that both the writing and the reading step use, and the reading step should check the file against it (Subsection 6.1).

9.3 All or nothing

A step must either complete and write all of its outputs, or fail with an error exit status and leave no output that looks complete. A truncated file, such as a table that ends after half of the samples, looks like a smaller valid result, and a workflow manager that finds it will consider the step done (Subsection 1.3).

Truncated outputs arise when a program crashes, is interrupted, or is killed for exceeding its memory or time limit while it writes. Write to a temporary file and give it its final name only after writing has succeeded, and let the workflow manager remove the outputs of failed steps where it offers to.

Unreported failures arise in shell code. By default, a shell script continues after a command fails, and a pipeline reports only the exit status of its last command, so a step can fail halfway and still exit with success. Run shell code in a strict mode, such as `set -euo pipefail` in Bash, and check the exit status of every program that a step calls.

10 Environments and tests

10.1 Pin what you run

Workflow code should run every step in an environment whose versions are pinned exactly: the language, the libraries, and every external tool. Record them in a lock file, in an environment file with exact versions, or in a container image identified by its digest rather than by a tag that can move.

The results of a workflow depend on the tools it calls as much as on its own code. A new release of a tool can change a default or fix a bug, and with it the numbers (Subsection 4.3). A workflow that installs whatever version is current can therefore give different results next month from the same inputs, the same configuration, and the same version of the workflow (Subsection 8.4).

This is the opposite of library code, which is installed next to other packages in the user's environment and therefore declares the range of versions it works with. Workflow code owns its program, and with it its environment. Steps may run in separate environments, for example when two tools need incompatible versions of the same dependency. Update pinned versions deliberately, as a change of the workflow that is tested (Subsection 10.2) and listed in the changelog (Subsection 4.4).

10.2 Test the whole workflow on a small dataset

Every workflow must have a test that runs it from start to finish on a small dataset, and the test should run automatically on every change (Subsection 8.1). Tests of the library code do not show that the steps fit together: that each step finds the files the previous one wrote, in the format it expects, and that every option of the configuration reaches the step that uses it. Many defects of workflow code are of this kind, and a run on real data reveals them only after hours, if at all.

The dataset should be small enough that the whole run takes minutes, and varied enough to exercise every step and every branch the configuration can select, including an input that fails or is skipped where the workflow allows that (Subsection 1.3). Check the outputs, not only that the run finished: compare them with known results, for example for simulated input, and check invariants such as the number of samples or the total of the counts (Subsection 8.2).

Part III

Library code

11 Structure

11.1 Design modules around clear responsibilities

Organize code into modules with clear, coherent responsibilities. Each module should own one part of the package, such as validation, computation, preprocessing, or visualization, and its name should say which. Avoid miscellaneous helper modules such as `utils` when a more specific name would describe the purpose of the code; a module that cannot be named precisely usually has no single responsibility.

Dependencies between modules should point in one direction, from the specific to the fundamental. Computation must not depend on visualization, and core computation should not depend on input and output. Knowledge shared by several modules, such as validation checks and numerical defaults, belongs in one module that the others import (Subsection 3.2).

11.2 No code at large

Library code lives in functions and classes. At module level, only imports, named constants (Subsection 3.3), and definitions belong. A statement that does work there runs on every import (Subsection 13.3), cannot be tested in isolation, and tends to communicate through global variables.

Prefer functions. Most scientific operations take data and parameters and return a result, which a function expresses directly and which is easy to test and compose. Use a class when there is state with invariants to protect, such as a fitted model, or a result that carries its values together with diagnostics and provenance (Subsection 8.7). A class that only groups functions, or stores its arguments for its methods to read back, should be a function.

11.3 Avoid plotting in core functionality

Plotting is brittle, backend-dependent, and parameter-heavy, so core functionality should not plot. If the package provides plotting, strictly separate computation from visualization: functions that compute return results, and separate functions plot them.

Plotting functions should take computed results as input and never recompute them. Where the plotting library supports it, they should draw into a target the caller can pass in, return the plot object instead of displaying it, and leave global plotting styles untouched (Subsection 13.1). The plotting library should be an optional dependency (Subsection 2.2).

12 Interfaces

12.1 Operate on data objects, not files

Library code should take data objects and return data objects; reading and writing files belongs to the caller or to workflow code. This avoids assumptions about file formats, directory layouts, and environments, and it prevents hidden side effects.

Accept the data types of the host stack, including lazy, memory-mapped, and chunked representations where the method permits it. Operating on data objects does not mean loading everything into memory; materializing such data behind the user's back is a surprise cost (Subsection 13.2).

Keeping input and output out of library code creates an obligation: the package's own objects, such as fitted models or result containers, should be storable with the host stack's standard serialization mechanisms, or convertible to standard types that are. If reading a domain-specific file format is itself part of the core domain, isolate it in a separate module that returns standard data objects.

12.2 Make the common case easy, and the uncommon case possible

A useful design principle for research software is to provide the API in *layers of abstraction*. Each layer exposes the same underlying capabilities but with different trade-offs between convenience and control.

Higher layers provide functions that solve common tasks with minimal user effort. These functions often make reasonable assumptions, provide sensible defaults, and combine multiple lower-level operations into a single call. They are designed for the typical use case.

Lower layers expose smaller, more specialized primitives. These functions require more explicit code from the user but provide greater flexibility, composability, and control over the computational workflow.

Each layer should be built from the public primitives of the layer below, so that the convenient functions are compositions of the controllable ones rather than a second implementation of the same knowledge (Subsection 3.2).

The relationship between abstraction layers, control, and convenience is illustrated in Figure 1. As abstraction layers increase, convenience typically increases while direct control decreases.

12.3 The library follows the caller's data types

If the caller passes data of a particular type, precision, or placement, keep it. Library code must not silently densify sparse data, upcast single to double precision, move data between the main processor and an accelerator, or convert one array type of the host stack into another. The output should have the same kind of type, precision, and placement as the input. Inputs that carry no such choice, such as a plain list of numbers, may be converted to the package's working type.

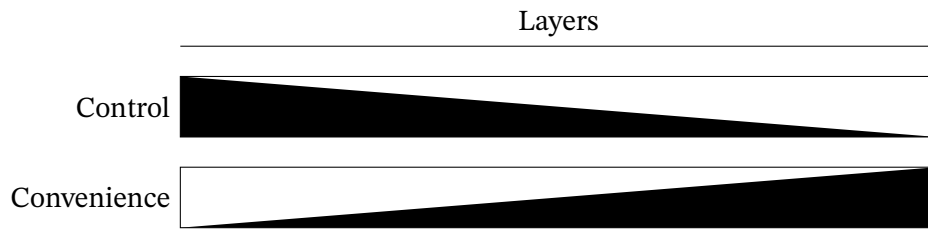


Figure 1: Layered API design. With increasing abstraction layers, direct control decreases while convenience increases.

The caller's choices reflect deliberate trade-offs of memory, speed, precision, and hardware. Following the incoming data preserves them; choosing for the user breaks the workflows built on them.

Correctness outranks this rule (Subsection 1.2). If the method cannot produce a correct result in the incoming precision, raise an error that names the required precision, rather than silently upcasting or silently returning a degraded result (Subsection 1.3); a user who wants the conversion can make it explicitly. Using higher precision for small intermediate quantities, such as an accumulator, is part of the method and must be documented as such (Subsection 4.5); converting the caller's data is not. If the package cannot operate on a given type at all, it should say so with an error rather than convert the data at hidden cost.

12.4 Keep validation proportionate

Validation has a cost, so the requirements for correctness and for cost (Subsection 13.2) can pull in different directions. Distinguish two kinds of check:

Metadata checks inspect only types, shapes, precision, declared units, and combinations of arguments. They are cheap and must always run in user-facing functions.

Data checks inspect the values themselves, for example for missing values, bounds, symmetry, or sortedness. They cost at least one pass over the data, and for data on an accelerator they can force the program to wait for the device.

Correctness outranks performance (Subsection 1.2), so data checks should run by default in user-facing functions, and their cost should be documented. A user who has already validated the data may turn them off through an explicit, clearly named argument. This follows the layered design of Subsection 12.2: the common case is safe, and the uncommon case is possible.

Data checks must not themselves create a surprise cost (Subsection 13.2). For lazy or chunked data, express a check as part of the same deferred computation, or apply it chunk by chunk.

13 No side effects

13.1 Do not touch global state

Library code must not change process-wide settings. It must not set or reseed the global random number generator, select hardware devices, change the number of threads used by numerical libraries, change floating-point error handling, change display options of data libraries, or rewrite plotting defaults. These settings belong to the user. A package that calls `set.seed` in R or `numpy.random.seed` in Python inside a function silently takes control of the user's session.

A change that is strictly scoped to the package's own computation, and restored before control returns to the caller, including when an error occurs, may be made where the host stack provides a mechanism for it. How the caller controls randomness instead is defined in Subsection 8.4.

13.2 No surprise cost

Time, memory, and network use are part of the contract. A helper must not, as a side effect, densify sparse data, materialize lazy data, download data, or transfer a whole array to or from an accelerator.

Expensive steps should be explicit in the function the user calls, or impossible to invoke by accident. A convenience wrapper that silently materializes the full dataset is a bug, not a feature. Input validation has costs as well; how they are balanced against correctness is defined in Subsection 12.4.

13.3 Importing the package has no side effects

Importing or loading the package must not download data, compile extensions, print, select backends, configure logging, or read from or write to the user's home directory. Users load libraries inside other tools, test suites, and notebooks; work done at load time is hidden work, and it slows down and destabilizes everything that depends on the package.

Put optional heavy setup behind an explicit function the caller invokes. Loading the package should be cheap and silent.

Part IV

Other code forms

Libraries and workflows are the backbone of most research software, but not its only forms. This part describes the most common other forms, and what each gains and loses compared with library and workflow code alone. The list is not exhaustive.

Inside, each form still consists of the two kinds of code. The part that owns the program, such as the event loop of a graphical application or the request handler of a web service, is workflow code, and the science it calls should be library code that works without it ([Library code and workflow code](#)). The form is then a front end: the same method can still be tested, scripted, and run at scale, and Parts I to III apply to its code as to any other.

14 Graphical applications

A graphical application is a standalone program that users operate through windows, menus, and dialogs, such as a viewer and annotation tool for microscopy images, software that controls a laboratory instrument, or a desktop tool for exploring spectra.

Advantages.

- It is the most accessible interface for humans. Users need no programming, and the interface can guide them through an analysis and show each result at once.
- It limits accidental damage. Users work further away from the source code, so they cannot change the method by mistake or call it in ways it was never meant to be called.
- It suits tasks that need human judgment at every step, such as drawing regions of interest or curating annotations.

Disadvantages.

- It is much more work to build and maintain. The interface code often outgrows the science, and graphical toolkits change between versions and operating systems.

- It is less flexible. Users are limited to what the interface offers, which is the other side of the second advantage, and every analysis carries the overhead of the interface, even when no human needs to look at it.
- It is poor for machines. A sequence of clicks cannot be scripted, scheduled on a cluster, or repeated for thousands of datasets.
- It is hard to test and to trace. Tests of an interface are slow and brittle, and a result obtained through dialogs leaves no record of its parameters unless the application writes one (Subsection 8.7).

15 Web applications and services

A web application runs on a server and is used through a browser, such as an interactive dashboard built with Shiny in R or Streamlit in Python, or an analysis platform such as Galaxy. A web service offers the same through a network interface that other programs call.

Advantages.

- Nothing has to be installed. Users need only a browser, and one deployment serves everyone with the same version.
- It brings users to data and computing power they do not have themselves, such as large reference databases or accelerators.
- A web service serves machines as well as humans, and from any programming language.

Disadvantages.

- Someone has to run it for as long as it exists: servers, security updates, user accounts, and costs. When the funding ends, the service often disappears, and with it the means to reproduce the results computed with it.
- Users cannot pin its version (Subsection 10.1). The service changes under them, so results have to record the version of the service that produced them (Subsection 8.7).
- Data have to travel to the server, which is a problem for large data and for sensitive data, such as patient records.

16 Notebooks

A notebook is a document that mixes code, its output, and prose, such as a Jupyter notebook or an R Markdown or Quarto document. In the terms of these guidelines, a notebook is workflow code: it owns the program and calls library code.

Advantages.

- Code, results, figures, and explanation sit in one document, which suits exploring data, explaining an analysis, and teaching.
- Feedback is immediate: run a cell and see the result.
- As examples in the documentation of a library, notebooks show users how to call it, in a form they can run.

Disadvantages.

- Cells can run in any order, so the output shown need not come from the code shown, and a notebook that worked yesterday may fail or give a different result when run from the top (Subsection 1.3). Run it from the top in a fresh session before trusting a result.

- Some formats, such as Jupyter's, store output and metadata together with the code, which makes notebooks hard to review and to compare between versions.
- Code in a notebook cannot be called from elsewhere, so it tends to be copied from notebook to notebook (Subsection 3.2). Code that a second notebook needs moves into library code, as for any workflow code.
- Running a notebook on a cluster, or for many inputs, needs extra tools.

17 Simulation codes

A simulation code is a large program that simulates a physical, chemical, biological, or social system, such as a climate model, a molecular dynamics engine, a fluid dynamics solver, or an epidemic model. It owns the program: it reads an input file that describes the simulation, runs a time-stepping or iterative loop, and writes the results, often in parallel on a cluster.

Advantages.

- One program can control memory layout, parallel execution, and input and output as a whole, and reach a performance that a chain of separate steps cannot.
- An input file describes a simulation completely. It can be versioned, shared, reviewed, and rerun, which helps reproducibility as long as the version of the code is recorded with it (Subsection 8.7).
- Established community codes carry years of validation that a new implementation would have to repeat.

Disadvantages.

- Its parts are hard to reuse. The science is often entangled with the main loop, the data structures, and the parallel layout, so no part can be called on its own, and users extend the code by editing its source or by writing routines for fixed hooks: the code is a framework, not a library ([Library code and workflow code](#)).
- Testing is expensive, because a realistic run takes hours on many processors. Tests need small, cheap configurations and invariants such as conservation laws (Subsection 8.2).
- Input files often have hundreds of options whose defaults change results (Subsection 4.3), and building the code depends on the compilers and libraries of a computing center, which makes runs hard to reproduce elsewhere.

18 Plugins

A plugin extends an existing application instead of standing alone, such as a plugin for an image analysis platform like Fiji or napari, for a geographic information system like QGIS, or for a network analysis tool like Cytoscape. The host application calls the plugin at fixed points.

Advantages.

- It reaches users where they already work. They need no new tool, and the plugin can use the interface, file formats, and visualization of the host instead of building its own.
- The host's plugin repository distributes and updates it.

Disadvantages.

- It is tied to the host. It can do only what the host's plugin interface allows, it breaks when that interface changes, and it lives only as long as the host does.

- A method written against the host's data model cannot be used outside the host, for example from a script or a workflow. Keep the plugin a thin adapter around library code that works without the host.
- Testing needs the host, or a stand-in for it.